
Task Arithmetic for Efficient Machine Unlearning: A Study on CNN Dynamics

May 29, 2026

Giulia Pietrangeli Lorenzo Musso

Abstract

This work evaluates *Task Arithmetic* for single-class unlearning on VGG-11, ResNet-18, and MobileNetV2. Standard metrics (ZRF, KL Divergence, MIA) point to successful knowledge removal. However, looking closer at the weight manipulation process, we found unexpected side-effects, specifically Semantic Shift and Over-forgetting. We use the **Anamnesis Index (AIN)** to quantify residual traces and the resulting “Rebound Effect”, showing that recoverable information was left behind.

1. Introduction

The “Right to be Forgotten”, introduced by regulations like the GDPR (European Parliament and Council, 2016), requires techniques to selectively erase data from trained neural networks. While retraining from scratch on the remaining data is the safest approach, its computational cost is often too high. Machine Unlearning aims to solve this problem by offering faster and more efficient methods for data removal.

In this project, we explore **Task Arithmetic** as a lightweight approach for selective unlearning in Convolutional Neural Networks (CNNs). We evaluate this method on the CIFAR-10 dataset using three diverse architectures: VGG-11 (purely sequential), ResNet-18 (residual connections), and MobileNetV2 (bottleneck). This selection allows us to analyze how different network topologies impact the stability and overall success of the unlearning process.

Our main contribution is evaluating how CNNs behave during weight manipulation. We assess data removal using standard evaluation metrics like Kullback-Leibler (KL) Divergence, Zero Retrain Forgetting (ZRF) Score, and Membership Inference Attacks (MIA). During this process, we highlight architectural side-effects such as *Semantic Shift*

and *Over-forgetting*. Furthermore, we use the Anamnesis Index (AIN) (Chundawat et al., 2023b) to measure latent residual traces and the “Rebound Effect”, which is defined as the model’s tendency to rapidly relearn the erased class. We also show how Task Arithmetic operates at a fraction of the cost of full retraining, making it an alternative for unlearning.

2. Related Work

Machine Unlearning and Weight Manipulation. Traditional unlearning methods often rely on computationally expensive retraining. Recently, *Task Arithmetic* (Ilharco et al., 2023) introduced a more direct approach: manipulating the weight space using “Task Vectors”, computed as the difference between fine-tuned and pre-trained weights. While it was originally proposed to steer the behavior of large foundation models, we adapt this framework to selectively erase specific classes. Also, while previous research mostly used weight manipulation for model compression (e.g., pruning (Han et al., 2015)), our work shifts the focus toward privacy-oriented data removal.

Architecture and Optimization. Existing research shows that a network’s structural design, such as the sequential layers in VGG (Simonyan & Zisserman, 2015), residual connections in ResNet (He et al., 2015), and bottlenecks in MobileNetV2 (Sandler et al., 2019), influences how it stores information in the latent space. Also, the choice of optimizer affects parameter stability. Standard optimizers couple weight decay with gradient updates, which can lead to uncontrolled weight growth. In contrast, decoupled optimizers (Loshchilov & Hutter, 2019) preserve the structural constraints of the weights, an important requirement for vectorial manipulation.

3. Methodology

3.1. Architectural Framework and Training Setup

To understand how network structure impacts Machine Unlearning, we analyze three diverse architectures: VGG-11 with Batch Normalization (sequential) (Simonyan & Zisserman, 2015; Ioffe & Szegedy, 2015), ResNet-18 (resid-

Email: G. Pietrangeli, L. Musso <{pietrangeli.2057291, musso.2049518}@studenti.uniroma1.it>.

ual) (He et al., 2015), and MobileNetV2 (bottleneck) (Sandler et al., 2019). We train all models from scratch on CIFAR-10. We also apply standard architectural modifications, such as replacing the initial 7×7 downsampling in ResNet-18 with a 3×3 convolution to properly accommodate the dataset’s 32×32 image resolution.

A design choice in our setup is using the **AdamW optimizer** (Loshchilov & Hutter, 2019). Standard adaptive optimizers often cause uncontrolled weight growth. We avoid this issue by using AdamW, which decouples weight decay from gradient updates. This keeps the weight space stable, ensuring that the task vectors capture actual concept changes instead of random magnitude variations. As a result, the vector subtraction becomes much more precise.

3.2. Expert Generation via Directional Shift

Our unlearning procedure requires a task-specific vector τ . We compute this by generating an “Expert Model”, which involves fine-tuning the pre-trained base model exclusively on the target class (Class 8: Ship). We selected this class because it is visually distinct from the dataset’s animal classes, helping to minimize noise from overlapping features. We must avoid the expert from losing its general knowledge, as this would corrupt the final task vector. To do this, we apply a **Directional Shift** strategy: we limit fine-tuning to 5 epochs and use a reduced learning rate (1×10^{-4}). This constraint ensures that the resulting task vector, calculated as $\tau = \theta_{ft} - \theta_{pre}$, captures a clean direction toward the target concept without damaging the model’s foundational convolutional filters.

3.3. Surgical Ablation and Hyperparameter Tuning

The intervention on the base model weights θ_{pre} is:

$$\theta_{unl} = \theta_{pre} + \alpha \cdot \text{mask}(\tau, \rho) \quad (1)$$

where α is the scaling factor (Ilharco et al., 2023) and ρ is the drop-percentile for magnitude-based pruning (Han et al., 2015). To optimize these parameters, we ran a **Grid Search**, testing $\alpha \in \{-0.5, -1.0, \dots, -3.0\}$ and sampling ρ across the $[0.50, 0.99]$ range using 20 linear steps.

Table 1 summarizes the optimal parameters and the resulting unlearning performance. Specifically, we report the Original Accuracy (Orig. Acc.) on the full dataset, the Retain Accuracy (Retain Acc.) on the remaining classes after the intervention, and the Forget Accuracy (Forget Acc.) on the target class. This allows us to properly evaluate the degradation before and after the unlearning. VGG-11 achieves perfect erasure: its target accuracy drops from 93.5% to 0.0%, while its retain accuracy only slightly decreases from the original 87.4% to 85.5%. This shows that sequential architectures can tolerate high-magnitude interventions ($\alpha = -2.5$) if applied to a highly sparse selection

of weights ($\rho = 0.990$).

Conversely, ResNet-18 and MobileNetV2 require more conservative parameters to unlearn the target class (with target accuracy falling from 95.0% to 3.0% and from 87.2% to 0.1%, respectively). However, because skip-connections and compressed bottlenecks are highly sensitive to weight changes, this process still causes an important drop in their retain accuracy (falling to 62.3% and 57.1%).

Table 1. Optimal hyperparameters (α, ρ) derived from the Grid Search, and performance comparison against the original base model accuracy.

Architecture	α	ρ (Drop)	Orig. Acc.	Retain Acc.	Forget Acc.
VGG-11 (bn)	-2.5	0.990	87.4%	85.5%	0.0%
ResNet-18	-1.0	0.629	89.1%	62.3%	3.0%
MobileNetV2	-1.5	0.500	77.1%	57.1%	0.1%

4. Experiments and Results

4.1. Performance Floor and Baselines

We benchmark Task Arithmetic (TA) against two standard optimization-based techniques: *Gradient Ascent* (GA) and *Random Labeling* (RL). As shown in Table 2, these traditional methods prove either overly disruptive or ineffective, depending on the model’s topology. In the sequential VGG-11, GA maximizes the loss causing a severe collapse of the global feature space (retain accuracy drops to 11.1%). However, applying RL to ResNet-18 protects the retain set (84.9%) but fails to completely unlearn the target concept, leaving a residual forget accuracy of 15.9%.

TA avoids these limitations through weight manipulation. Thanks to magnitude-based vector subtraction, TA achieves perfect erasure (0.0%) in VGG-11 while preserving 85.5% of the retain set. Notably, MobileNetV2 suffers from generalized degradation across all three methods. This suggests that highly compressed models lack the redundancy needed to handle targeted weight changes without losing general accuracy.

Table 2. Comparison: Task Arithmetic (TA) vs Optimization-based Baselines.

Architecture	Task Arithmetic		Grad. Ascent		Rand. Labeling	
	Retain	Forget	Retain	Forget	Retain	Forget
ResNet-18	62.3%	3.0%	46.9%	2.8%	84.9%	15.9%
VGG-11 (bn)	85.5%	0.0%	11.1%	0.0%	47.9%	0.0%
MobileNetV2	57.1%	0.1%	49.6%	4.4%	59.3%	9.6%

4.2. Privacy Metrics and Validation

To assess whether the models successfully removed the target data, we evaluate their latent distributions using **KL Divergence** and **ZRF Score** (Chundawat et al., 2023a) (distance from a randomly initialized network).

As summarized in Table 3, VGG-11 achieves a very low KL divergence and a high ZRF score, indicating its predictive distribution is statistically indistinguishable from a model that never learned the target class. Conversely, ResNet-18 and MobileNetV2 exhibit significant KL divergence and notably lower ZRF values. This disparity highlights the phenomenon of “Fake Unlearning”: despite their low target accuracy, internal feature correlations remain structured in non-sequential architectures. In other words, they aren’t truly deleting the data, just hiding it. **Membership Inference Attacks (MIA)** (Shokri et al., 2017) obtain near-random AUCs (~ 0.50) across all models, confirming that target membership can no longer be extracted.

Table 3. Privacy Metrics. Optimal unlearning is characterized by low KL divergence, high ZRF score, and MIA AUC near 0.50.

Architecture	KL Div. ($\downarrow$)	ZRF Score ($\uparrow$)	MIA AUC
VGG-11 (bn)	0.97	0.83	~ 0.44
ResNet-18	3.20	0.32	~ 0.43
MobileNetV2	5.91	0.41	~ 0.47

4.3. Architectural Side-Effects: Semantic Shift and Over-forgetting

The t-SNE and Confusion Matrix plots in Appendix A show exactly how each architecture handles data removal.

Semantic Shift. Confusion matrices show that predictions for the erased class do not become purely random, but are instead routed to specific alternatives. In VGG-11, the “Ship” concept shifts mostly toward “Truck”, suggesting the model relies on basic visual shapes. In ResNet-18, predictions surprisingly shift toward “Cat”. This indicates that residual connections bypass the suppressed filters and redirect signals to other dominant feature extractors. MobileNetV2 shifts unpredictably toward “Dog”, highlighting its fragility.

Over-forgetting. This occurs when removing a target concept accidentally degrades the model’s accuracy on visually similar, untargeted classes. To measure this collateral damage, we calculate the accuracy gap between the similar “Airplane” class and the other remaining classes. VGG-11 isolates the concept well, showing only a small 3.4% gap (82.5% Airplane vs 85.9% Others). In contrast, ResNet-18 suffers a much larger 13.3% gap (50.5% vs 63.8%), showing that residual connections entangle features across simi-

lar classes, making precise removal difficult. MobileNetV2 also registers an 8.9% gap (49.2% vs 58.1%), confirming it is highly sensitive to local weight changes.

4.4. The Rebound Effect and Computational Efficiency

To quantify residual traces of the deleted class, we evaluate the **Anamnesis Index (AIN)** (Chundawat et al., 2023b). Defined as $AIN = E_{unt}/E_{comp}$, it compares the epochs required for the unlearned model to relearn the target knowledge versus a model learning it from scratch. An $AIN < 1.0$ highlights the *Rebound Effect*, meaning the unlearned weights retain enough information to relearn the concept faster than a baseline model.

VGG-11 shows the strongest rebound ($AIN = 0.36$), recovering the target concept in just 4 epochs compared to the 11 baseline epochs. MobileNetV2 also relearns quickly ($AIN = 0.53$, converging in 16 vs. 30 epochs), while ResNet-18 shows a more moderate rebound ($AIN = 0.75$, 9 vs. 12 epochs). These results prove that across different architectures, weight manipulation leaves hidden feature representations that the optimizer can rapidly reactivate.

Computational Efficiency. All experiments were timed on a MacBook Air (Apple M4 chip, 16GB RAM). The framework achieves an 88.27% computational reduction, requiring 381.36 seconds for VGG-11 versus 3251.29 seconds for full retraining. While generating the expert model requires a 378.98-second cost, the actual unlearning step executes in just 2.39 seconds via a simple vector subtraction (Ilharco et al., 2023). This makes the method a possible practical alternative for real-world unlearning requests.

5. Discussion and Conclusions

Task Arithmetic enables efficient machine unlearning, reducing computations by 88.27% versus retraining. However, effectiveness strictly depends on architecture. Sequential models (VGG-11) approximate baseline networks with minimal over-forgetting. Conversely, residual (ResNet-18) and bottleneck (MobileNetV2) topologies resist precise removal, inducing “Fake Unlearning”: they aren’t truly deleting the data, just hiding it, which causes unintended feature entanglement.

Also, the Anamnesis Index (AIN) shows that complete erasure remains unreachable. Vector subtraction leaves residual traces, triggering a *Rebound Effect* that allows rapid concept relearning. Ultimately, Task Arithmetic is a practical solution for unlearning, but you have to sacrifice true irreversibility to get that computational speed.

6. Statement on the use of AI

During the development of this project, Artificial Intelligence (AI) was utilized as a support tool. Its application was confined to the following activities:

- **Literature Review and State of the Art (SOTA):** Assistance in navigating recent scientific literature regarding Machine Unlearning. Specifically, building upon our pre-existing familiarity with the Task Arithmetic framework (derived from a prior NLP project), AI was employed to research and evaluate alternative ablation methodologies and comparative metrics, thereby enriching the theoretical background (e.g., the concept of Zero Retrain Forgetting).
- **Validation and Feasibility Analysis:** Critical brainstorming to assess the temporal and computational feasibility of proposed ideas. This helped filter out excessively demanding tests given the available hardware, favoring more targeted and efficient experimental setups and metrics.
- **Debugging and Low-Level Troubleshooting:** Assistance in diagnosing PyTorch bottlenecks and crashes. Specific interventions included resolving a memory allocation error (Segmentation Fault) related to hardware acceleration on Apple MPS chips, and fixing a Batch Normalization layer crash caused by singleton batches during dynamic dataset filtering.
- **Technical Consultation and Parametric Choices:** Utilizing AI as a sounding board to overcome experimental impasses. Specifically, we consulted the AI to: (1) Evaluate the opportunity to expand the search space for the hyperparameter α , having observed that the VGG-11 architecture was saturating at the established lower bounds (-2.0); (2) Discuss the calibration of the Learning Rate during re-learning tests (Anamnesis Index). The AI validated our hypothesis to drastically lower the LR to prevent instantaneous re-acquisition, enabling precise measurement of latent residual traces.
- **Data Analysis and Visualization:** Assistance in the comparative analysis of the obtained metrics and support in writing the code (e.g., Matplotlib/Seaborn) required to generate appropriate visual representations. The AI facilitated the graphical translation of numerical results, particularly for plotting latent space clusters (t-SNE) and generating side-by-side confusion matrices.
- **Academic Revision and Terminology:** Support during the drafting phase for lexical optimization, ensuring the adoption of a rigorous technical vocabulary aligned with Deep Learning research standards.

Overall, AI was used strictly as a support tool to speed up our workflow.

References

- Chundawat, V. S., Tarun, A. K., Mandal, M., and Kankanhalli, M. Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher, 2023a. URL <https://arxiv.org/abs/2205.08096>.
- Chundawat, V. S., Tarun, A. K., Mandal, M., and Kankanhalli, M. Zero-shot machine unlearning. *IEEE Transactions on Information Forensics and Security*, 18: 2345–2354, 2023b. ISSN 1556-6021. doi: 10.1109/tifs.2023.3265506. URL <http://dx.doi.org/10.1109/TIFS.2023.3265506>.
- European Parliament and Council. Regulation (eu) 2016/679 on the protection of natural persons with regard to the processing of personal data (general data protection regulation), 2016. URL <https://gdpr-info.eu/art-17-gdpr/>. Art. 17 (Right to erasure).
- Han, S., Pool, J., Tran, J., and Dally, W. J. Learning both weights and connections for efficient neural networks, 2015. URL <https://arxiv.org/abs/1506.02626>.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., and Farhadi, A. Editing models with task arithmetic, 2023. URL <https://arxiv.org/abs/2212.04089>.
- Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015. URL <https://arxiv.org/abs/1502.03167>.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks, 2019. URL <https://arxiv.org/abs/1801.04381>.
- Shokri, R., Stronati, M., Song, C., and Shmatikov, V. Membership inference attacks against machine learning models, 2017. URL <https://arxiv.org/abs/1610.05820>.

Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition, 2015. URL <https://arxiv.org/abs/1409.1556>.

Appendix

A. Visual Proofs of Topological Side-Effects

To facilitate reproducibility, the complete source code is publicly available at: <https://github.com/giulapietrangeli/Task-Arithmetic-Machine-Unlearning.git>.

This appendix provides the qualitative visual evidence for the Semantic Shift and latent space reconfiguration discussed in Section 4.

A.1. t-SNE Latent Space Visualizations

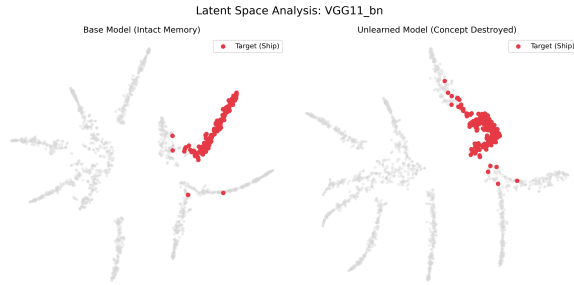


Figure 1. t-SNE latent space visualization for VGG-11 post-unlearning. The red cluster represents the target class (Ship), which is infiltrating nearby different clusters.

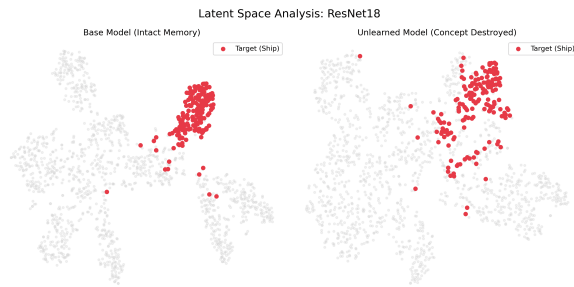


Figure 2. t-SNE latent space visualization for ResNet-18 post-unlearning. The red cluster representing the target class (Ship) is fragmented and infiltrates adjacent distributions.

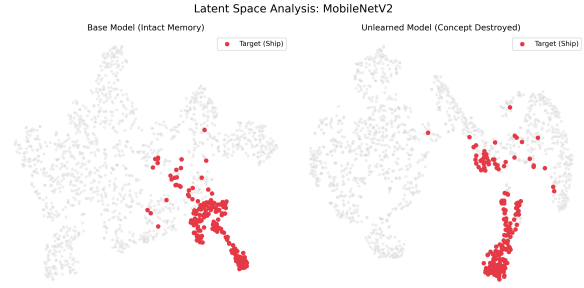


Figure 3. t-SNE latent space visualization for MobileNetV2 post-unlearning. The target class (Ship) remains mostly isolated but is translated to a peripheral region.

A.2. Confusion Matrices

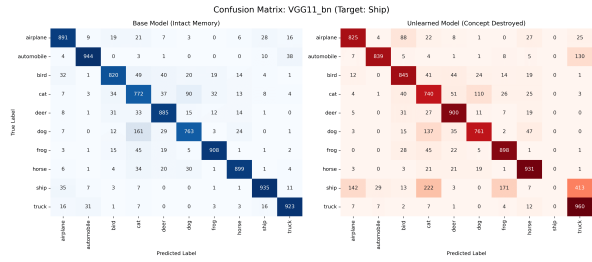


Figure 4. Confusion Matrix for VGG-11 post-unlearning. Note the deterministic semantic shift, with the majority of "Ship" instances misclassified as "Truck".

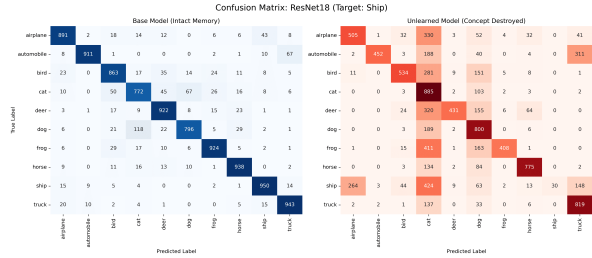


Figure 5. Confusion Matrix for ResNet-18 post-unlearning. The pronounced and unexpected shift of "Ship" predictions toward "Cat" illustrates the complex and unpredictable latent signal generated by residual connections when a target concept is ablated.

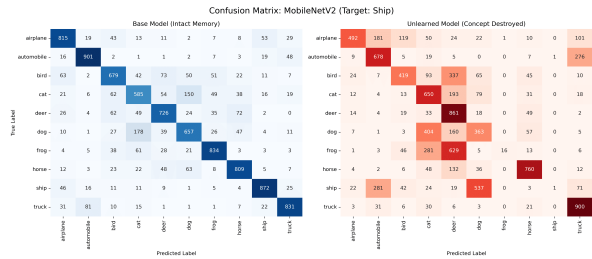


Figure 6. Confusion Matrix for MobileNetV2 post-unlearning. The chaotic redirection of predictions toward unrelated classes like "Dog" visually corroborates the architectural collapse.